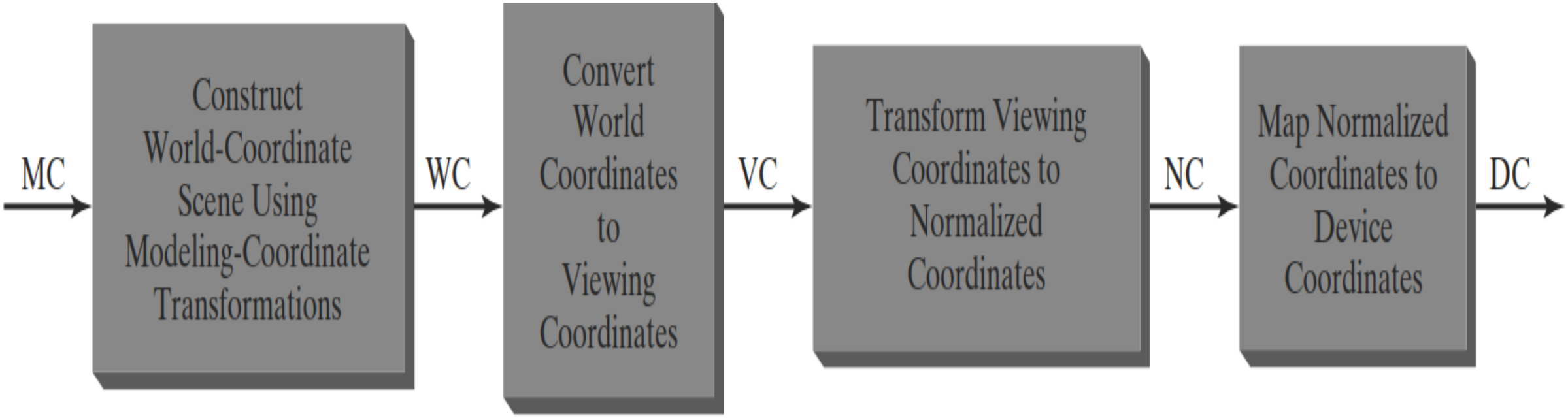
Computer Graphics and Man-Machine Interface

Dr. Manar Donia

CMPn322 - CMPN322 - CMP 422



The **two-dimensional viewing transformation** maps world-coordinate scenes to device coordinates, often referred to as the **window-to-viewport transformation**.

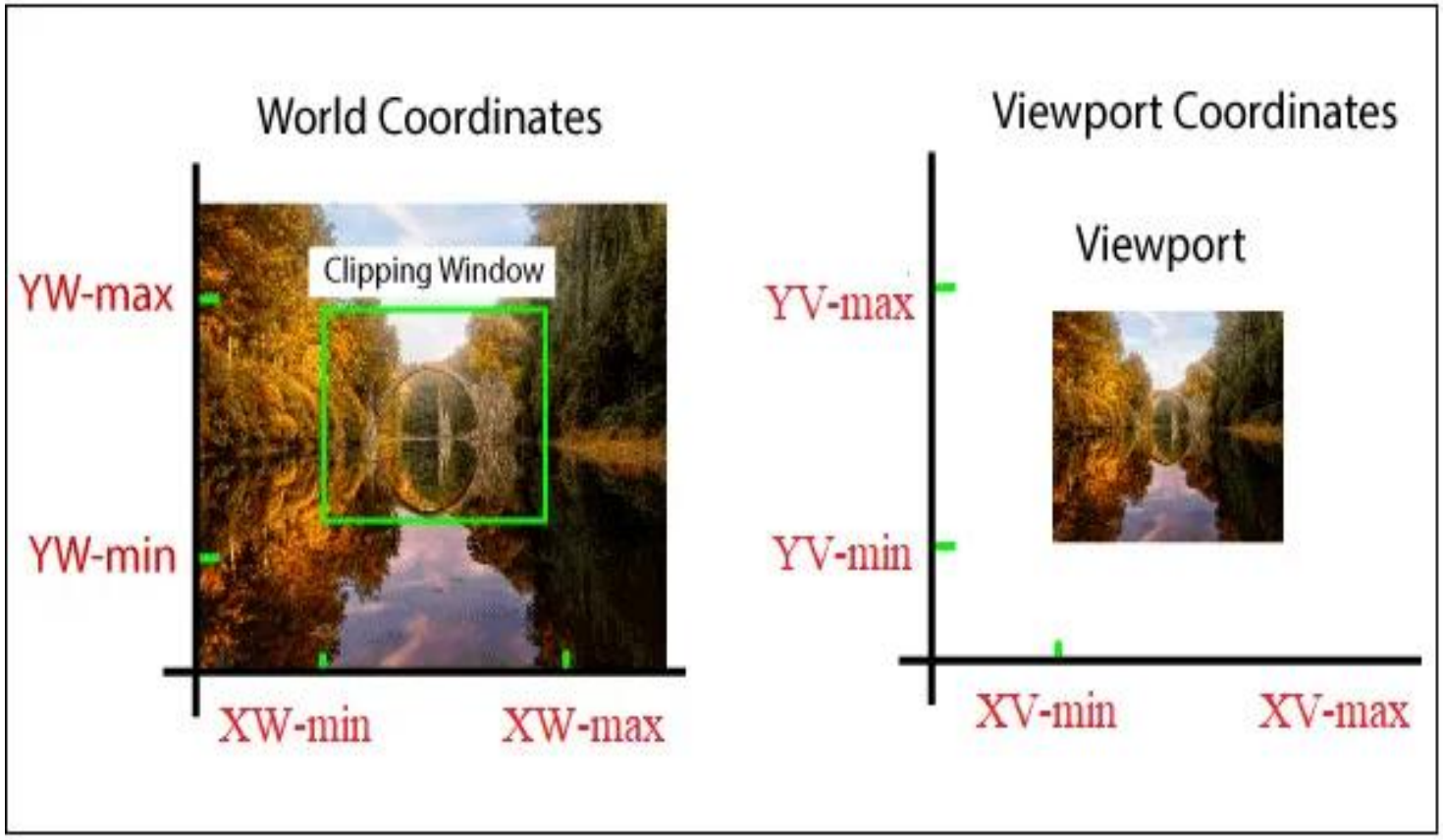
- This process primarily involves converting clipping-window coordinates to viewport coordinates.
  - Once a world-coordinate scene has been constructed, we could set up a separate two dimensional, viewing-coordinate reference frame for specifying the clipping window
  - To ensure device independence, graphics systems convert object descriptions into **normalized coordinates** (ranging from 0 to 1 or -1 to 1).
  - The viewport is then defined in either normalized or screen coordinates, and its

Window to Viewport Mapping

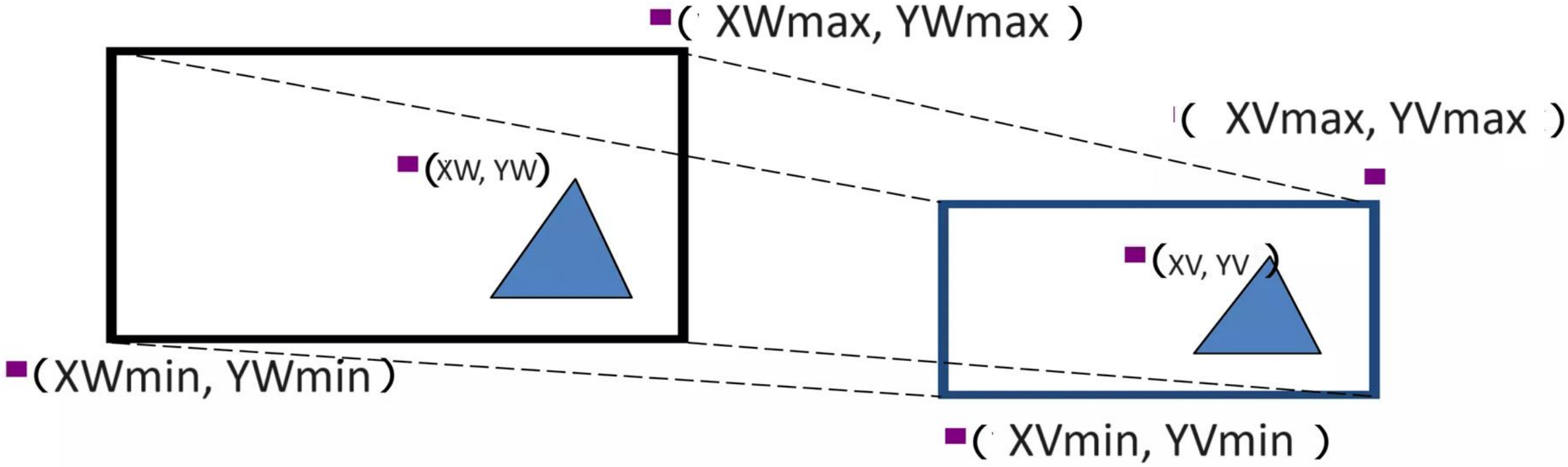
Window to Viewport Mapping :is the process of mapping or transforming a 2D world coordinate scene to device coordinates.

Window is What to be displayed

Viewport is where to be displayed



Window to Viewport Mapping



Some mathematical computations are required to map window and viewport

As shown the size of window is greater than size of viewport, we need to decrease the size of window according to the size of viewport .

Window to viewport Mapping

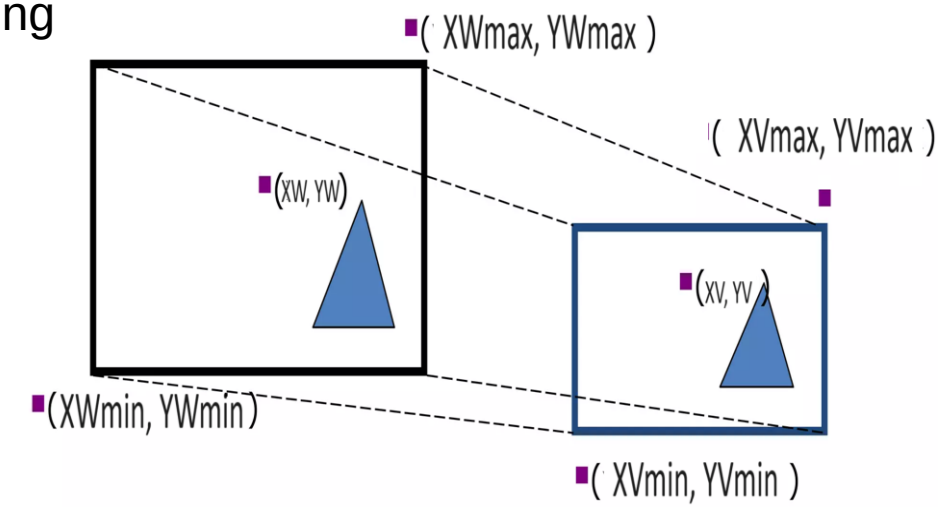
Suppose we have a point (XW, YW) and we want to find the corresponding viewport coordinates point (XV, YV)

If the relative position is same so

$$\frac{XW - XWmin}{XWmax - XWmin} = \frac{XV - XVmin}{XVmax - XVmin}$$

And

$$\frac{YW - YWmin}{YWmax - YWmin} = \frac{YV - YVmin}{YVmax - YVmin}$$



$$XV - XVmin = \frac{(XW - XWmin) * (XVmax - XVmin)}{XWmax - XWmin}$$

$$XV - XVmin = \left(\frac{XVmax - XVmin}{XWmax - XWmin}\right) * XW - XWmin$$

Scaling factor S_X

$$YV - YVmin = \frac{(YW - YWmin) * (YVmax - YVmin)}{YWmax - YWmin}$$

$$YV - YVmin = \left(\frac{YVmax - YVmin}{YWmax - YWmin}\right) * (YW - YWmin)$$

Scaling factor S_y

$$XV = XVmin + (S_X * (XW - XWmin))$$

$$YV = YVmin + (S_y * (YW - YWmin))$$

Example

Consider the window and viewport. The window has (W.l, W.r, W.b, W.t) = (20,80, 40, 80) and the viewport has (V.l, V.r, V.b, V.t) = (30,60,40,60) for (XW, YW)=(30,80) FIND (XV ,YV)

ANSWER:

Window width→ W.r- W.l & Window hight→ W.t- W.b

Viewport width→ V.r- V.l & viewport hight→ V.t- V.b

XWmin=20 Xvmin=30 XWmax=80 Xvmax=60

YWmin=40 YVmin=40 YWmax=80 Yvmax=60

Sx= $\frac{XVmax-XVmin}{XWmax-XWmin}$= = $\frac{60-30}{80-20} = 0.5$

$Sy = \frac{YVmax-YVmin}{YWmax-YWmin} = 0.5$

XV=30+(0.5*(30-20))=35

YV=40+(0.5*(80-40))=60

XV= $XVmin$+(S_X* $(XW - XWmin)$)

YV= $YVmin$+(S_y *$(YW - YWmin)$)

Example

- Consider the window and viewport The window has
- (W.l, W.r, W.b, W.t) = (0,2, 0, 1) and the viewport has (V.l, V.r, V.b, V.t) = (40, 400, 60, 300). For (XW, YW)=(1,0.5) FIND (XV ,YV)

ANSWER:

XWmin=0 Xvmin=40 XWmax=2 Xvmax=400

YWmin=0 YVmin=60 YWmax=1 Yvmax=300

$Sx= \frac{XVmax-XVmin}{XWmax-XWmin} = \frac{400-40}{2-0} = 180$

$Sy = \frac{YVmax-YVmin}{YWmax-YWmin} = \frac{300-60}{1-0} = 240$

XV= $XVmin+(S_X * (XW - XWmin))$

YV= $YVmin+(S_y *(YW - YWmin))$

XV=40+(180*(1-0))=220

YV=60+(240*(0.5-

Doing it in OpenGL.
OpenGL makes it very easy to use the window to viewport mapping: it automatically passes each vertex it is given (via a `glVertex2*()` command) through a sequence of transformations that carry out the desired mapping. It also automatically clips off parts of objects lying outside the world window. All we need do is to set up these transformations properly, and OpenGL does the rest.

For 2D drawing the world window is set by the function `gluOrtho2D()`, and the viewport is set by the function `glViewport()`. These functions have prototypes:

```
void gluOrtho2D(GLdouble left, GLdouble right, GLdouble bottom, GLdouble
top);
```

which sets the window to have lower left corner (`left`, `bottom`) and upper right corner (`right`, `top`), and

```
void glViewport(GLint x, GLint y, GLint width, GLint height);
```

which sets the viewport to have lower left corner (`x`, `y`) and upper right corner (`x + width`, `y + height`).

By default the viewport is the entire screen window: if *W* and *H* are the width and height of the screen window, respectively, the default viewport has lower left corner at (0, 0) and upper right corner at (*W*, *H*).

- FIND (XV ,YV) For (XW, YW)=(1.5,0.8)
- Consider the window and viewport. The window has (W.l, W.r, W.b, W.t) = (0,2, 0, 1) and the viewport has (V.l, V.r, V.b, V.t) = (40, 400, 60, 300).

ANSWER:

XWmin=0 Xvmin=40 XWmax=2 Xvmax=400

YWmin=0 YVmin=60 YWmax=1 Yvmax=300

$Sx = \frac{XVmax - XVmin}{XWmax - XWmin} = \frac{400-40}{2-0} = 180$

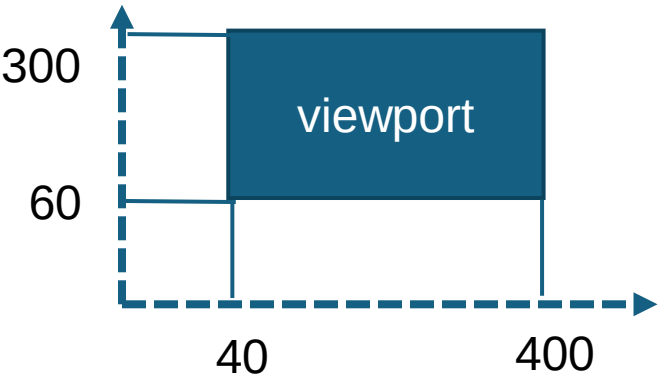
$Sy = \frac{YVmax - YVmin}{YWmax - YWmin} = \frac{300-60}{1-0} = 240$

XV= $XVmin + (S_x * (XW - XWmin))$

YV= $YVmin + (S_y * (YW - YWmin))$

XV=40+(180*(1.5-0))=310

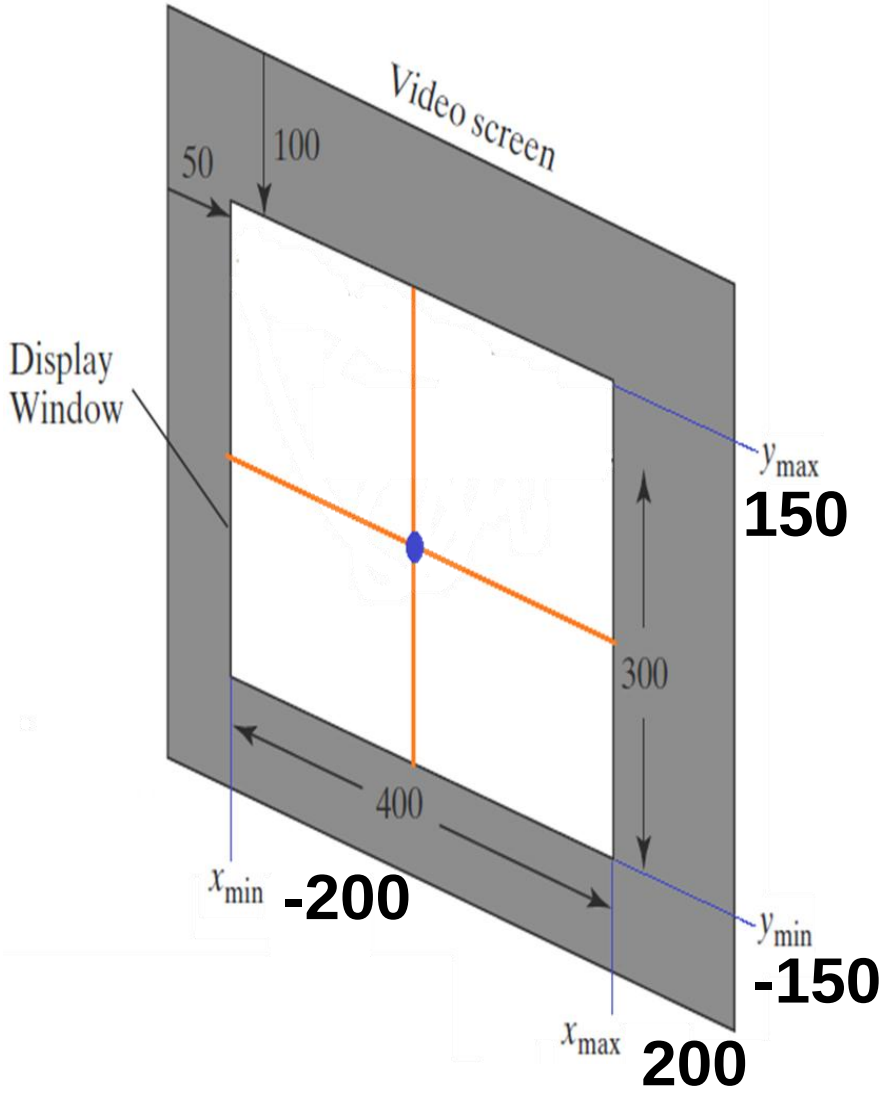
YV=60+(240*(0.8-0))=252




```
glMatrixMode(GL_PROJECTION);
glLoadIdentity();
gluOrtho2D(0.0, 2.0, 0.0, 1.0);      // sets the window
glViewport(40, 60, 360, 240);        // sets the viewport
```

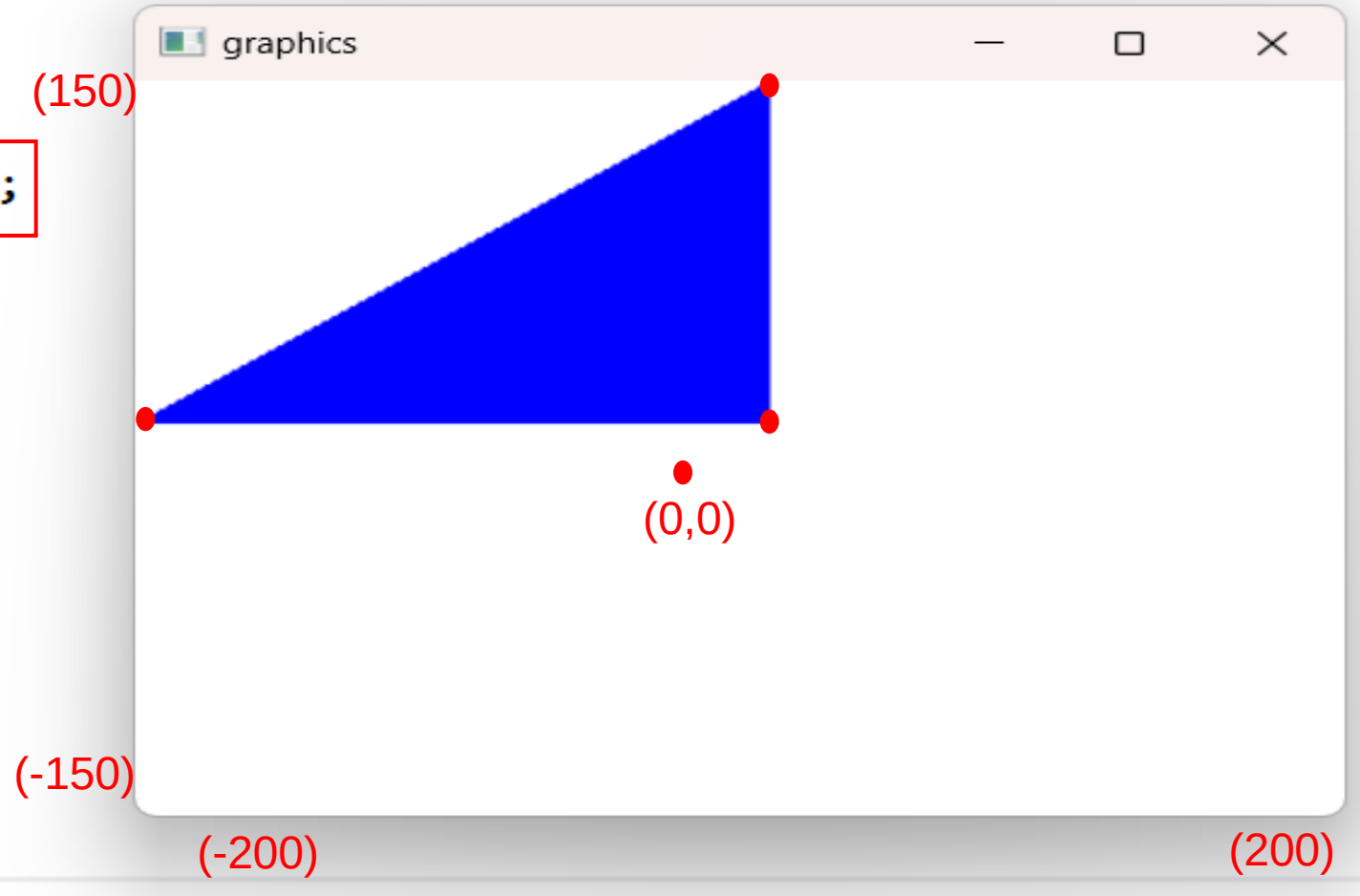
DISPLAY-WINDOW MANAGEMENT

```
int main(int argc, char** argv)
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_RGB);//set display mode
    glutInitWindowSize(400, 300);//set window size
    glutInitWindowPosition(100, 150);//set window position
    glutCreateWindow("graphics");
    myInit();
    glutDisplayFunc(myDisplay);
    glutReshapeFunc(reshape);
    Timer(0);
    glutMainLoop();
    return 0;
}
void myInit()
{
    glClearColor(1.0, 1.0, 1.0, 0.0);
    glColor3f(0.0f, 0.0f, 0.0f);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(-200,200,-150,150);
}
```



DISPLAY-WINDOW MANAGEMENT

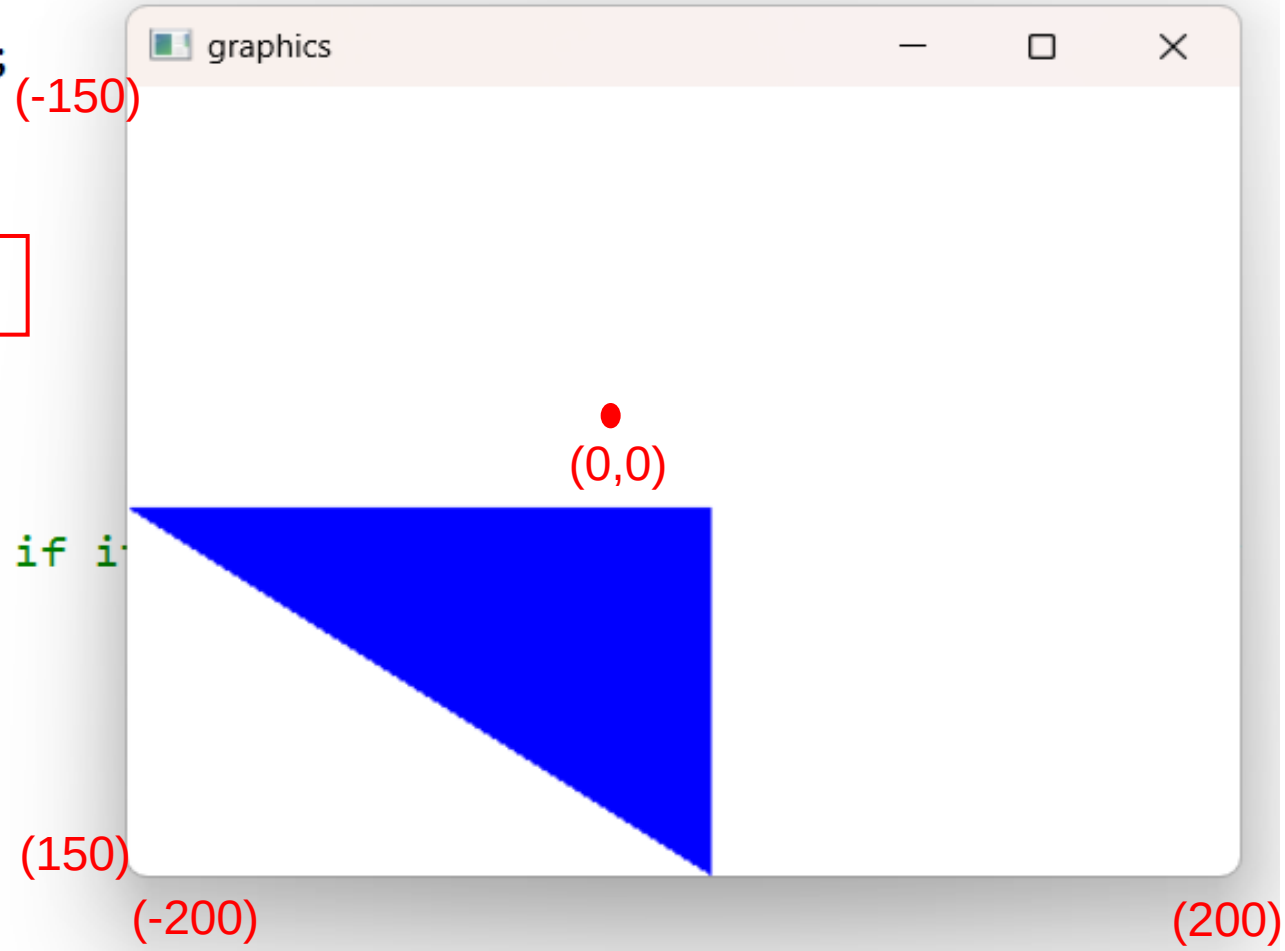
```
void myInit()
{
    glClearColor(1.0, 1.0, 1.0, 0.0);
    glColor3f(0.0f, 0.0f, 0.0f);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(-200,200,-150,150);
}
void myDisplay()
{
    glClear(GL_COLOR_BUFFER_BIT);
    glPointSize(5);
    glColor3f(0.0, 0.0, 1.0);
    glBegin(GL_TRIANGLES);
    glVertex2f(-200, 10);
    glVertex2f(10,10);
    glVertex2f(10, 150);
    glEnd();
    glFlush();
}
```



DISPLAY-WINDOW MANAGEMENT

```
void myInit()
{
    glClearColor(1.0, 1.0, 1.0, 0.0);
    glColor3f(0.0f, 0.0f, 0.0f);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(-200,200,150,-150);
}

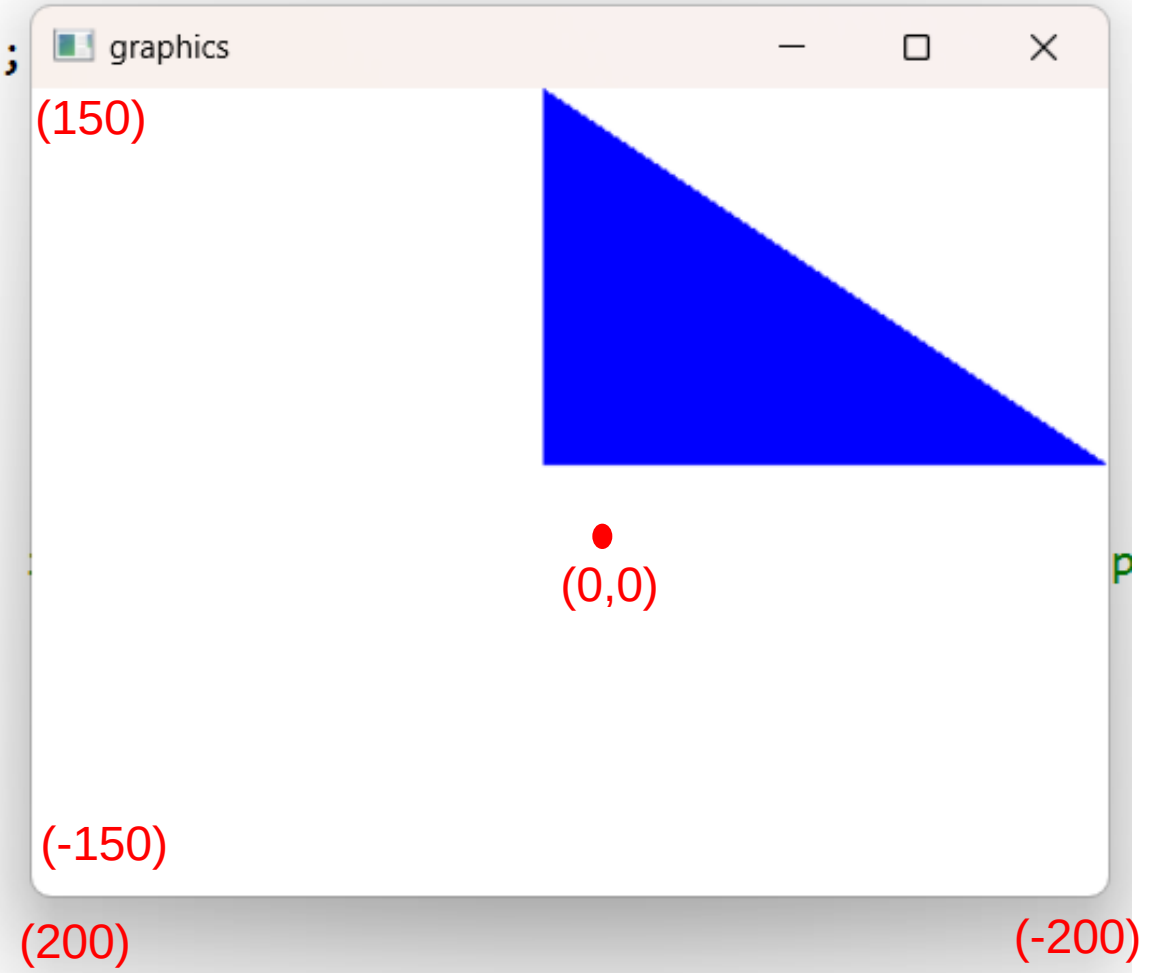
void myDisplay()
{
    glClear(GL_COLOR_BUFFER_BIT); // if i
    glPointSize(5);
    glColor3f(0.0f, 0.0f, 1.0f);
    glBegin(GL_TRIANGLES);
    glVertex2f(-200, 10);
    glVertex2f(10, 10);
    glVertex2f(10, 150);
    glEnd();
    glFlush();
}
```



DISPLAY-WINDOW MANAGEMENT

```
void myInit()
{
    glClearColor(1.0, 1.0, 1.0, 0.0);
    glColor3f(0.0f, 0.0f, 0.0f);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(200,-200,-150,150);
}

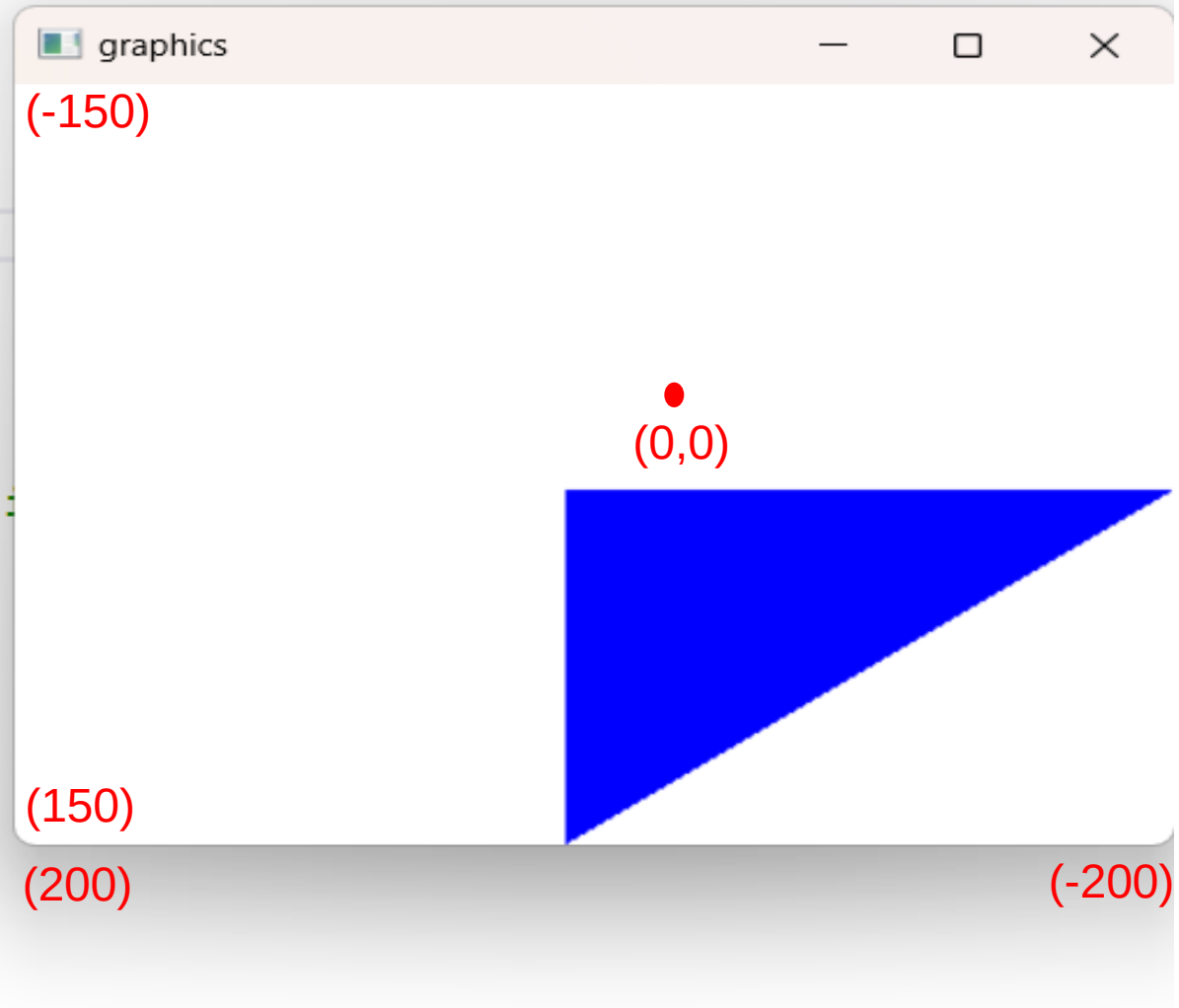
void myDisplay()
{
    glClear(GL_COLOR_BUFFER_BIT); //
    glPointSize(5);
    glColor3f(0.0f, 0.0f, 1.0f);
    glBegin(GL_TRIANGLES);
    glVertex2f(-200, 10);
    glVertex2f(10, 10);
    glVertex2f(10, 150);
    glEnd();
    glFlush();
}
```



DISPLAY-WINDOW MANAGEMENT

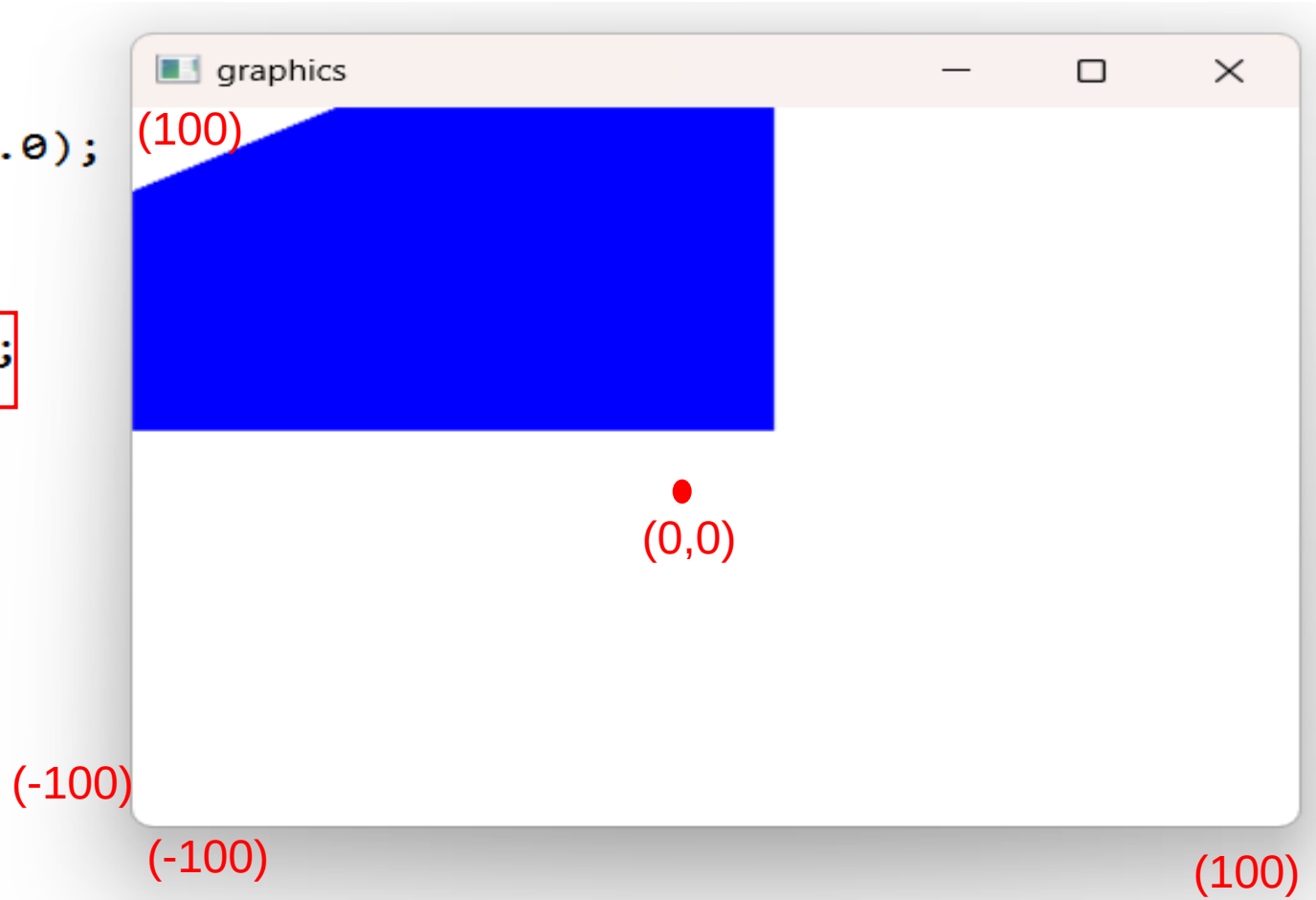
```
void myInit()
{
    glClearColor(1.0, 1.0, 1.0, 0.0);
    glColor3f(0.0f, 0.0f, 0.0f);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(200,-200,150,-150);
}

void myDisplay()
{
    glClear(GL_COLOR_BUFFER_BIT); // 
    glPointSize(5);
    glColor3f(0.0f, 0.0f, 1.0f);
    glBegin(GL_TRIANGLES);
    glVertex2f(-200, 10);
    glVertex2f(10, 10);
    glVertex2f(10, 150);
    glEnd();
    glFlush();
}
```



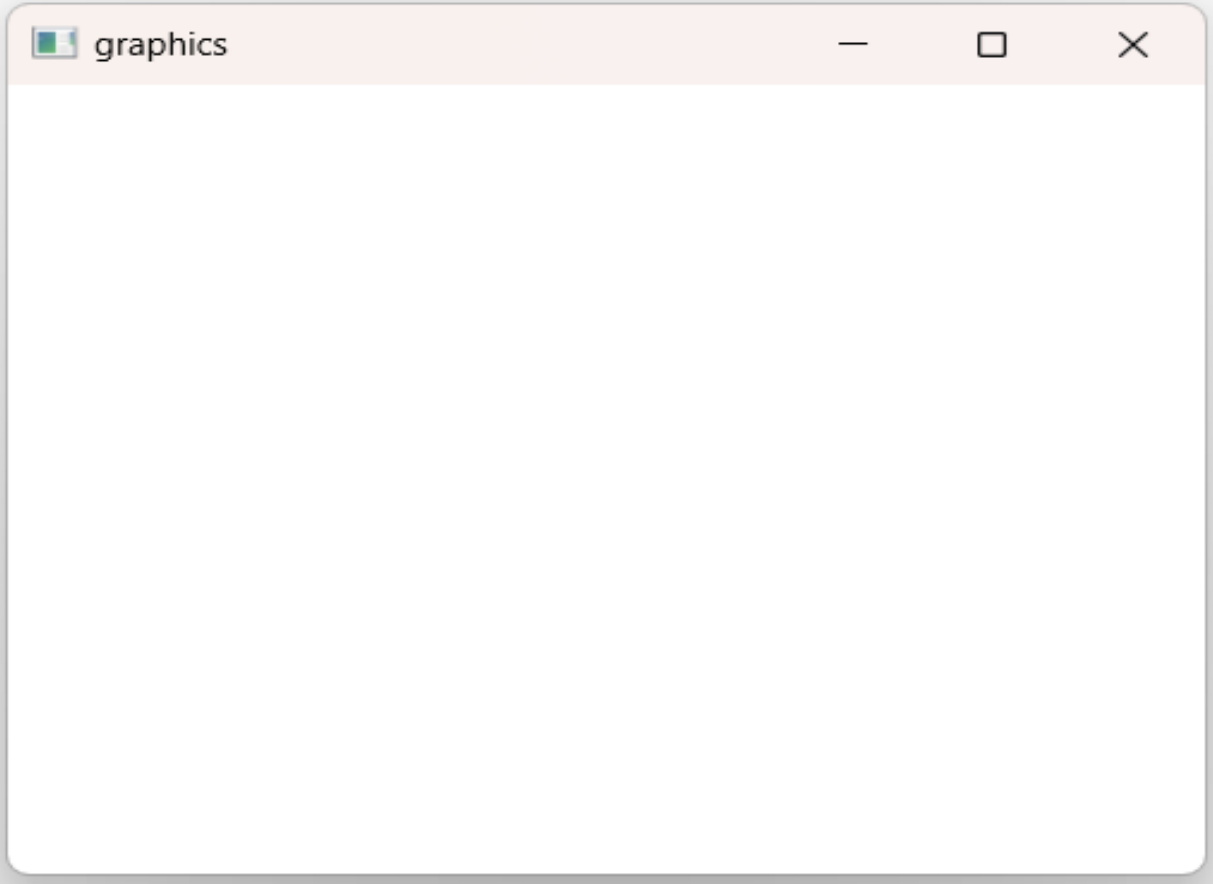
DISPLAY-WINDOW MANAGEMENT

```
void myInit()
{
    glClearColor(1.0, 1.0, 1.0, 0.0);
    glColor3f(0.0f, 0.0f, 0.0f);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(-100,100,-100,100);
}
void myDisplay()
{
    glClear(GL_COLOR_BUFFER_BIT);
    glPointSize(5);
    glColor3f(0.0, 0.0, 1.0);
    glBegin(GL_TRIANGLES);
    glVertex2f(-200, 10);
    glVertex2f(10,10);
    glVertex2f(10, 150);
    glEnd();
    glFlush();
}
```



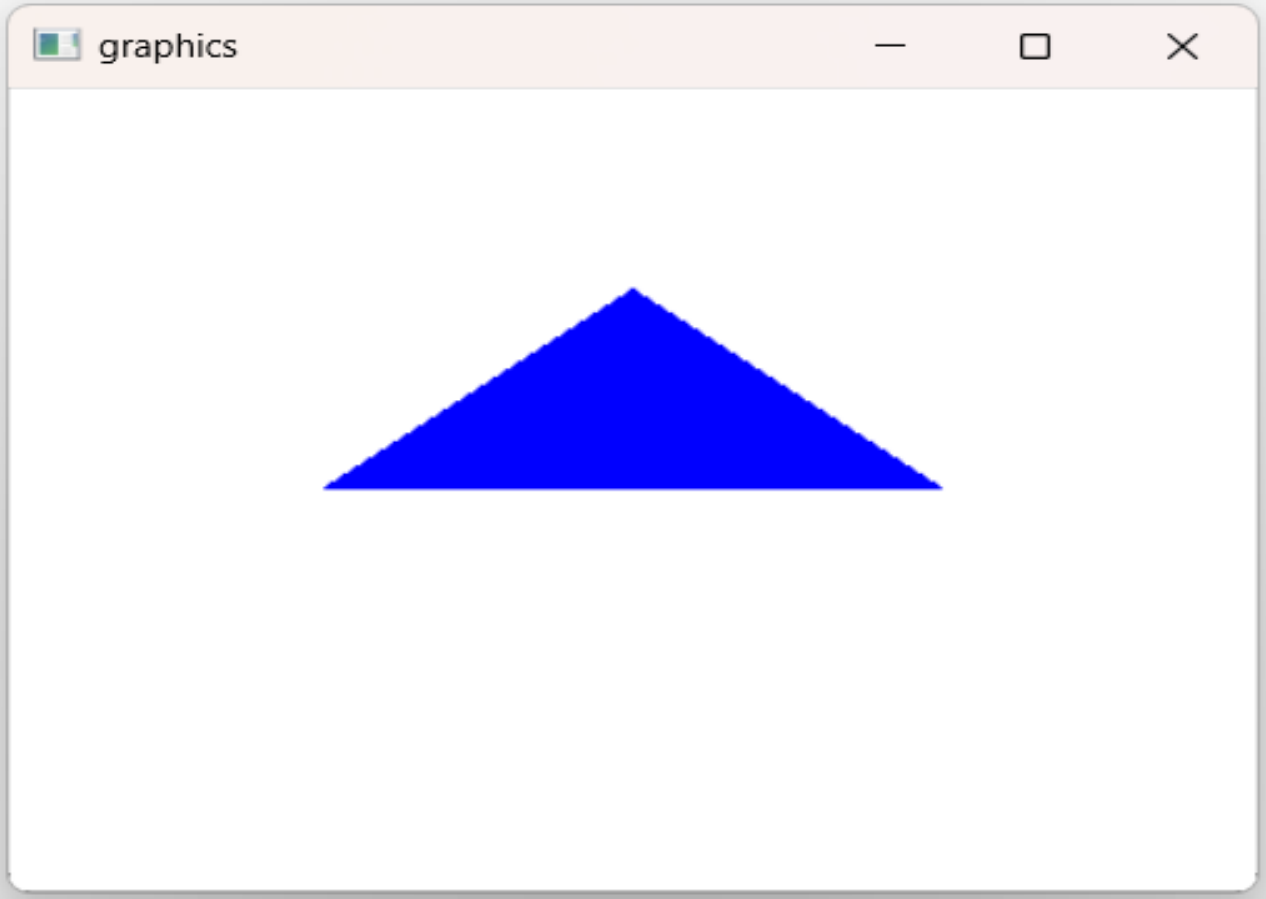
Can I draw a simple figure without **gluOrtho2D?**

```
void myInit()
{
    glClearColor(1.0, 1.0, 1.0, 0.0);
    glColor3f(0.0f, 0.0f, 0.0f);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    //gluOrtho2D(-100,100,-100,100);
}
void myDisplay()
{
    glClear(GL_COLOR_BUFFER_BIT);
    glPointSize(5);
    glColor3f(0.0, 0.0, 1.0);
    glBegin(GL_TRIANGLES);
    glVertex2f(-200, 10);
    glVertex2f(10,10);
    glVertex2f(10, 150);
    glEnd();
    glFlush();
}
```



Can I draw a simple figure without **gluOrtho2D?**

```
void myInit()
{
    glClearColor(1.0, 1.0, 1.0, 0.0);
    glColor3f(0.0f, 0.0f, 0.0f);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    //gluOrtho2D(-100,100,-100,100);
}
void myDisplay()
{
    glClear(GL_COLOR_BUFFER_BIT);
    glPointSize(5);
    glColor3f(0.0, 0.0, 1.0);
    glBegin(GL_TRIANGLES);
    glVertex2f(-0.5,0);
    glVertex2f(0,0.5);
    glVertex2f(0.5,0);
    glEnd();
    glFlush();
}
```



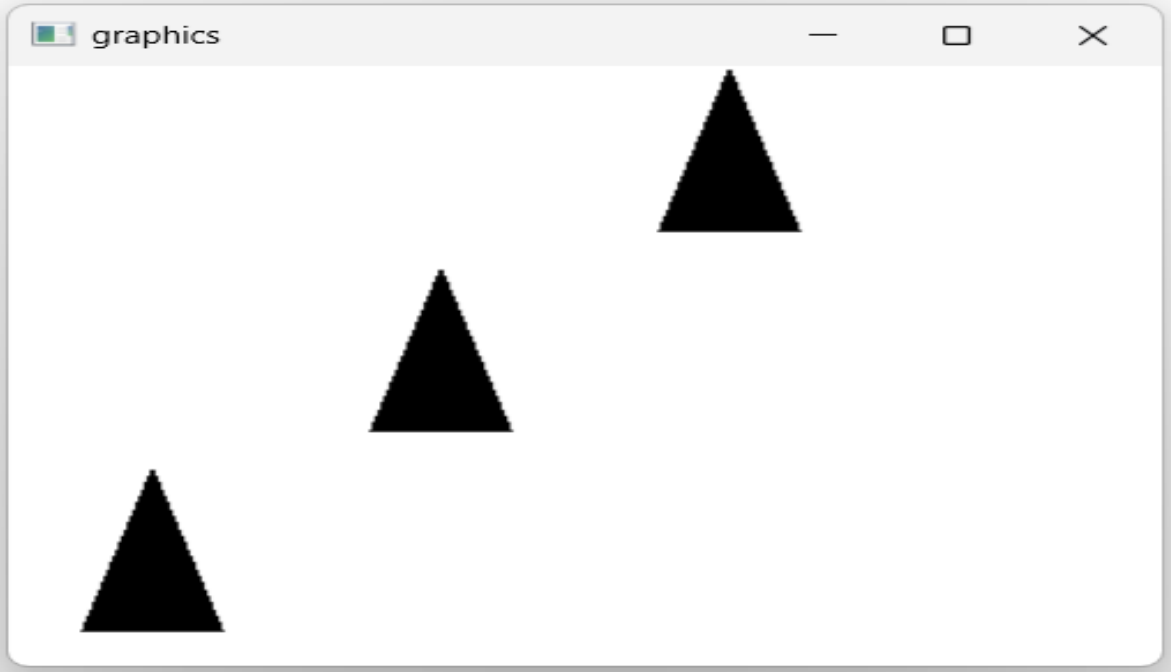
Create multiple viewports where each viewport have different scene

```
void myDisplay()
{
    glClear(GL_COLOR_BUFFER_BIT);
    glPointSize(5);
    glViewport(0, 0, 100, 100);
    glBegin(GL_TRIANGLES);
    glVertex2f(-100, -100);
    glVertex2f(100, -100);
    glVertex2f(0, 150);
    glEnd();

    glViewport(100, 100, 100, 100);
    glBegin(GL_TRIANGLES);
    glVertex2f(-100, -100);
    glVertex2f(100, -100);
    glVertex2f(0, 150);
    glEnd();

    glViewport(200, 200, 100, 100);
    glBegin(GL_TRIANGLES);
    glVertex2f(-100, -100);
    glVertex2f(100, -100);
    glVertex2f(0, 150);
    glEnd();

    glFlush();
}
```



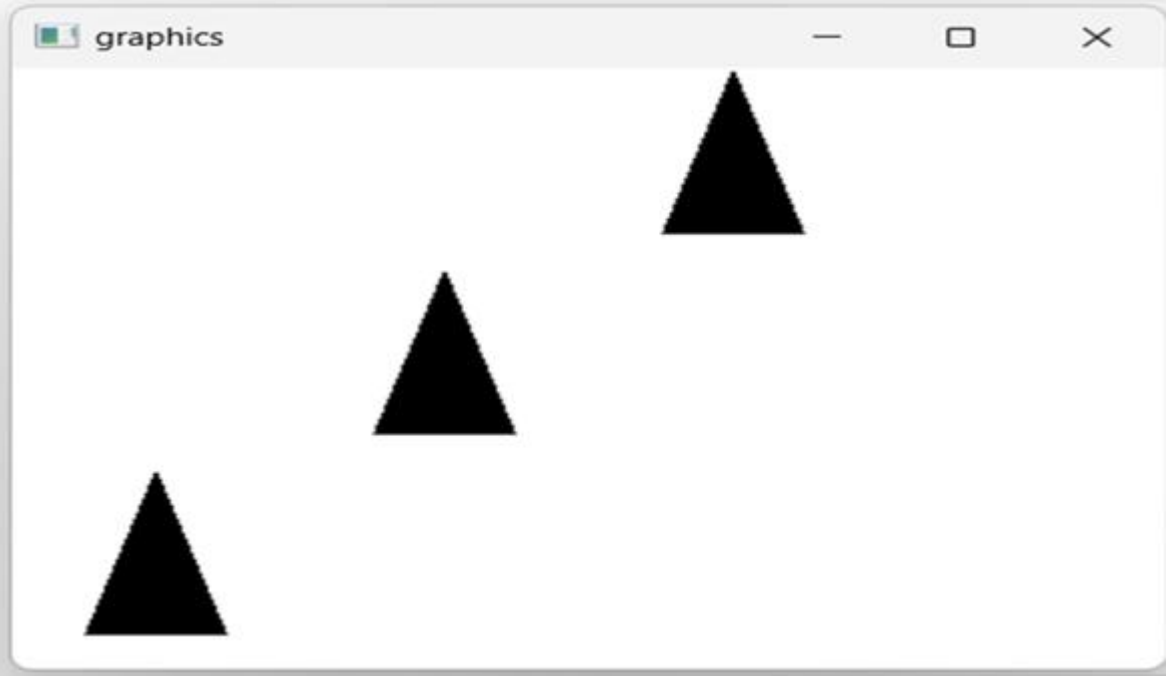
Create multiple viewports where each viewport have different scene

```
void myDisplay()
{
    glClear(GL_COLOR_BUFFER_BIT);
    glPointSize(5);
    glColor3f(0.0f, 0.0f, 0.0f);

    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(-200, 200, -150, 150);
    glViewport(0, 0, 100, 100);
    glBegin(GL_TRIANGLES);
    glVertex2f(-100, -100);
    glVertex2f(100, -100);
    glVertex2f(0, 150);
    glEnd();

    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(-200, 200, -150, 150);
    glViewport(100, 100, 100, 100);
    glBegin(GL_TRIANGLES);
    glVertex2f(-100, -100);
    glVertex2f(100, -100);
    glVertex2f(0, 150);
    glEnd();

    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(-200, 200, -150, 150);
    glViewport(200, 200, 100, 100);
    glBegin(GL_TRIANGLES);
    glVertex2f(-100, -100);
    glVertex2f(100, -100);
    glVertex2f(0, 150);
    glEnd();
    glFlush();
}
```



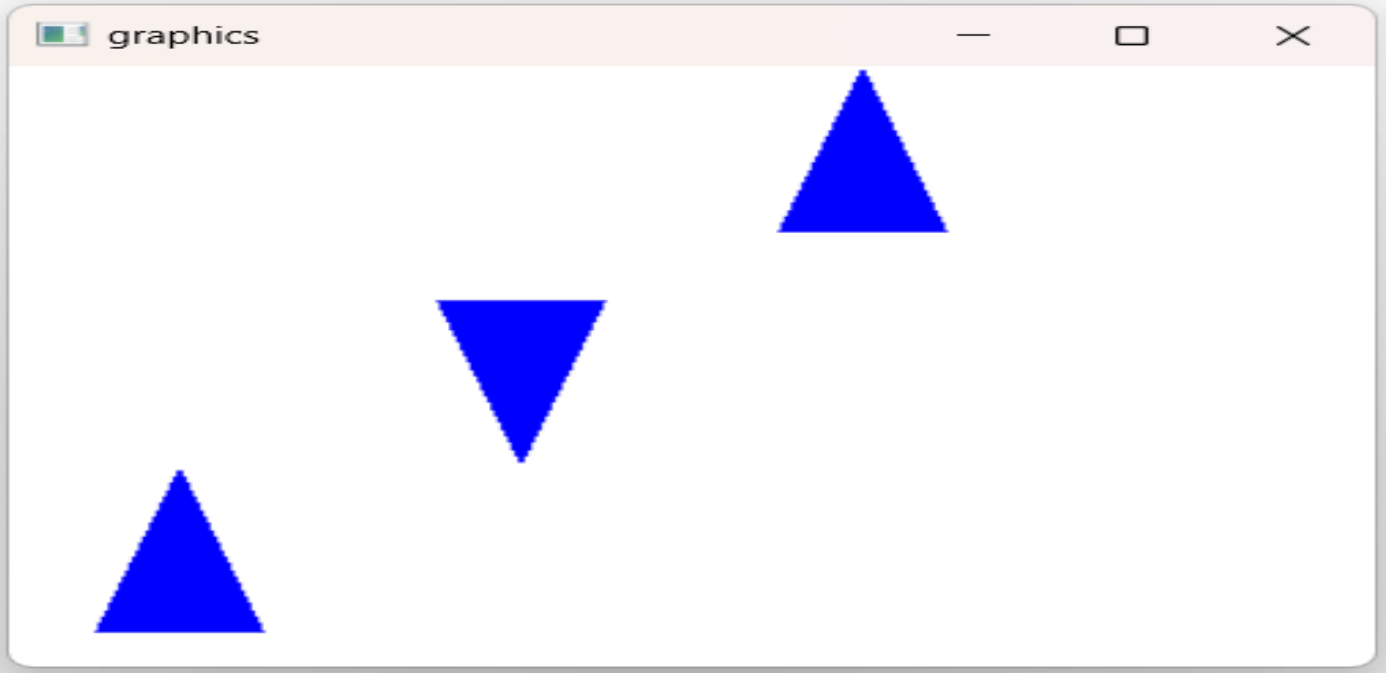
Create multiple viewports where each viewport have different scene

```
void myDisplay()
{
    glClear(GL_COLOR_BUFFER_BIT);
    glPointSize(5);
    glColor3f(0.0f, 0.0f, 1.0f);

    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(-200, 200, -150, 150);
    glViewport(0, 0, 100, 100);
    glBegin(GL_TRIANGLES);
    glVertex2f(-100, -100);
    glVertex2f(100, -100);
    glVertex2f(0, 150);
    glEnd();

    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(-200, 200, 150, -150);
    glViewport(100, 100, 100, 100);
    glBegin(GL_TRIANGLES);
    glVertex2f(-100, -100);
    glVertex2f(100, -100);
    glVertex2f(0, 150);
    glEnd();

    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(-200, 200, -150, 150);
    glViewport(200, 200, 100, 100);
    glBegin(GL_TRIANGLES);
    glVertex2f(-100, -100);
    glVertex2f(100, -100);
    glVertex2f(0, 150);
    glEnd();
    glFlush();
}
```



Setting the Window and Viewport Automatically

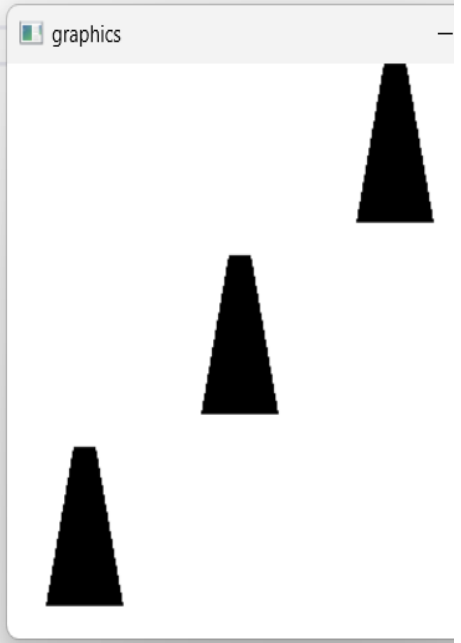
- Setting window
  - Often the programmer does not know where or how big the object of interest lies in world coordinates. **The object might be stored in a file or it might be generated procedurally by some algorithm whose details are not known.** In such cases it is convenient to let the application determine a good window to use.
- The usual approach is to find a window that includes the entire object: to achieve this the object's extent must be found. The extent (or bounding box) of an object is the aligned rectangle that just covers it.

```
void myDisplay()
{
    glClear(GL_COLOR_BUFFER_BIT);
    glPointSize(5);
    glViewport(0, 0, 100, 100);
    glBegin(GL_TRIANGLES);
    glVertex2f(-100, -100);
    glVertex2f(100, -100);
    glVertex2f(0, 250);
    glEnd();

    glViewport(100, 100, 100, 100);
    glBegin(GL_TRIANGLES);
    glVertex2f(-100, -100);
    glVertex2f(100, -100);
    glVertex2f(0, 250);
    glEnd();

    glViewport(200, 200, 100, 100);
    glBegin(GL_TRIANGLES);
    glVertex2f(-100, -100);
    glVertex2f(100, -100);
    glVertex2f(0, 250);
    glEnd();

    glFlush();
}
```



Setting the Window and Viewport Automatically

How can the extent be computed for a given object? If all the endpoints of its lines are stored in an array `pt[i]`, for $i = 0, 2, ..., n - 1$ the extent can be computed by finding the extreme values of the x- and y- coordinates in this array. For instance, the left side of the extent is the smallest of the values `pt[i].x`. Once the extent is known, the window can be made identical to it.

If, on the other hand, an object is procedurally defined, there may be no way to determine its extent ahead of time. In such a case the routine may have to be run twice:

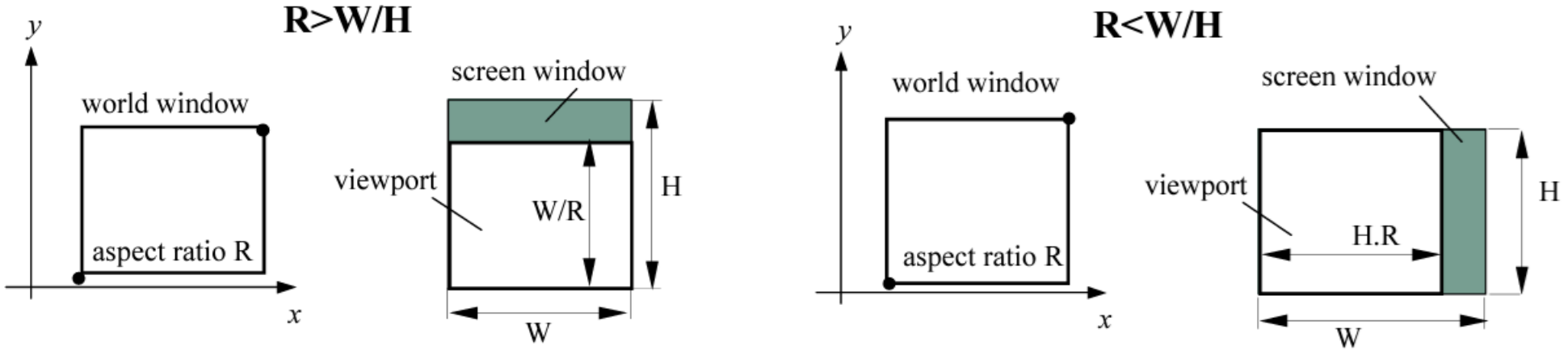
Pass 1: Execute the drawing routine, but do no actual drawing; just compute the extent. Then set the window.
Pass 2: Execute the drawing routine again. Do the actual drawing.

DISPLAY-WINDOW MANAGEMENT

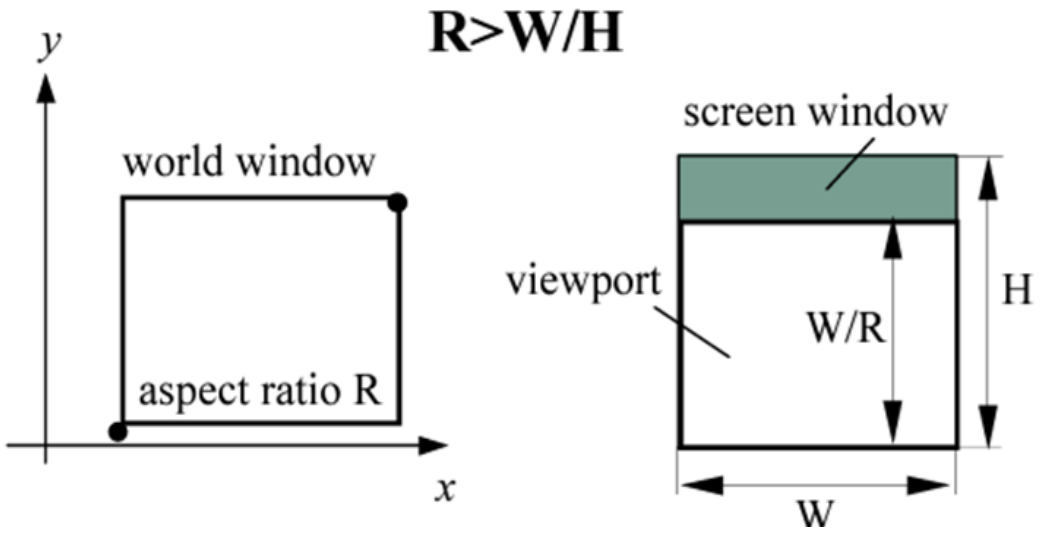
```
int main(int argc, char** argv)
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_RGB);//set display mode
    glutInitWindowSize(400, 300);//set window size
    glutInitWindowPosition(100, 150);//set window position
    glutCreateWindow("graphics");
    myInit();
    glutDisplayFunc(myDisplay);
    glutReshapeFunc(reshape);
    Timer(0);
    glutMainLoop();
    return 0;
}
```

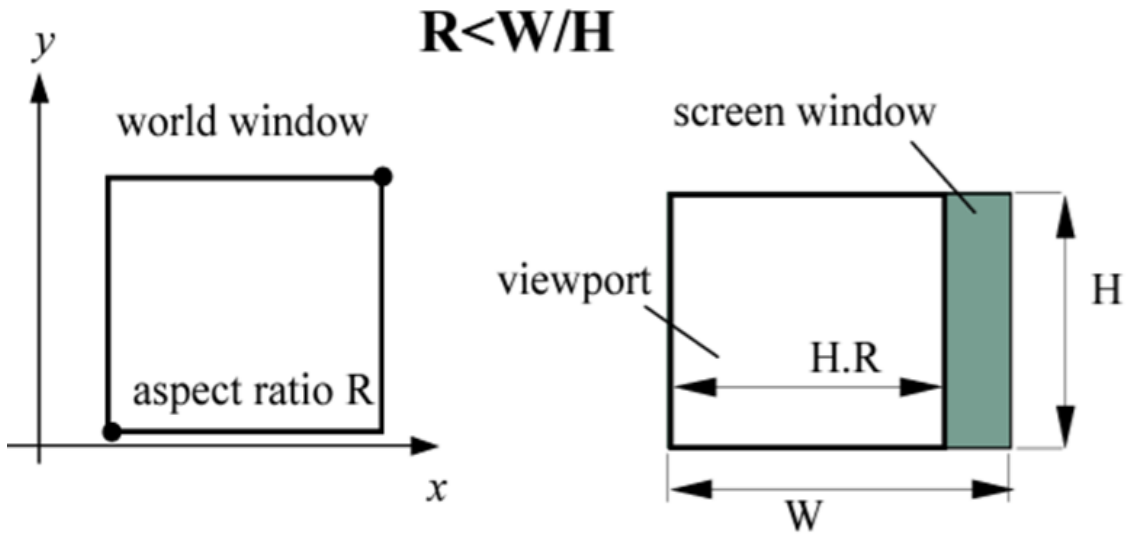
Setting the Window and Viewport Automatically

- Automatic setting of the viewport to Preserve Aspect Ratio
- Suppose you want to draw the largest undistorted version of a figure that will fit in the screen window. For this you need to specify a viewport that has the same aspect ratio as the world window. A common wish is to find the largest such viewport that will fit inside the screen window on the display
  - R= Aspect Ratio of world
  - Two situations to be considered:



Setting the Window and Viewport Automatically





- Therefore, at largest the viewport will have width W and height W/R

glViewport(0,0,W,W/R);

$$\frac{v_w}{v_h} = \frac{v_w}{v_{w \div R}} = \frac{v_w}{v_w} * R = R$$

- Therefore, at largest the viewport will have width H*R and height H

glViewport(0,0,H*R,H);

$$\frac{v_w}{v_h} = \frac{v_{h*R}}{v_h} = R$$

Example

Find the matching viewport for a world window with aspect ratio R=1.6 when the screen window has width 360 and height 200.

Answer

R= 1.6

$$\frac{W_w}{H_h} = \frac{360}{200} = 1.8$$

R< $\frac{W_w}{H_w}$ → therefore H_v=H=200 and W_v= H*R= $200 * 1.6 = 320$

glViewport(0,0,320,200);

Example

Find the matching viewport for a window with aspect ratio R=2 when the screen window has width 360 and height 200.

Answer

R= 2

$$\frac{W_v}{H_v} = \frac{360}{200} = 1.8$$

R > $\frac{W_v}{H_v}$ → therefore W_v=W=360 and H_v= W/R= $360/2 = 180$

glViewport(0,0,360,180);

Thank You